

Repaso de demostraciones

Inducción, correctitud y complejidad asintótica

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2^{do} Cuatrimestre de 2026

Agenda de hoy

- 1 ¿Qué es una demostración?
- 2 ¿Cómo demostramos?
- 3 Repaso de complejidad asintótica.
- 4 Inducción.
- 5 Correctitud de algoritmos recursivos.
- 6 Otras herramientas: contrarrecíproco, contradicción, palomar, mínimo elemento.
- 7 Tips + Conclusiones.

¿Qué es una demostración?

Una **demostración matemática** es un argumento convincente sobre la veracidad de una proposición matemática.

¿Qué es una demostración?

Una **demostración matemática** es un argumento convincente sobre la veracidad de una proposición matemática.

Inmediatamente tenemos la pregunta: ¿convence *a quién*?

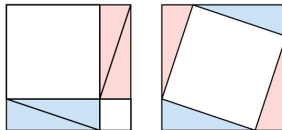
¿Qué es una demostración?

Una **demostración matemática** es un argumento convincente sobre la veracidad de una proposición matemática.

Inmediatamente tenemos la pregunta: ¿convence *a quién*?

Teorema (Pitágoras)

Dado un triángulo rectángulo, la suma de los cuadrados de los catetos es igual al cuadrado de la hipotenusa.



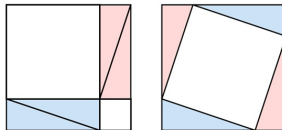
¿Qué es una demostración?

Una **demostración matemática** es un argumento convincente sobre la veracidad de una proposición matemática.

Inmediatamente tenemos la pregunta: ¿convince *a quién*?

Teorema (Pitágoras)

Dado un triángulo rectángulo, la suma de los cuadrados de los catetos es igual al cuadrado de la hipotenusa.



¿Esto es siquiera una demostración? ¿Los convence? ¿Cómo saben, o definen, si «está bien»?

¿A quién queremos convencer?

- Una demostración *heurística* (« $n! + 1$ no es divisible por nadie debajo de n , luego es primo») puede convencer a un apurado... y ser falsa: $4! + 1 = 25 = 5^2$.

¿A quién queremos convencer?

- Una demostración *heurística* (« $n! + 1$ no es divisible por nadie debajo de n , luego es primo») puede convencer a un apurado... y ser falsa: $4! + 1 = 25 = 5^2$.
- Una demostración totalmente formal (por ejemplo, escrita en el lenguaje de programación Lean 4) convence a una computadora, pero es ilegible para un par humano.

¿A quién queremos convencer?

- Una demostración *heurística* (« $n! + 1$ no es divisible por nadie debajo de n , luego es primo») puede convencer a un apurado... y ser falsa: $4! + 1 = 25 = 5^2$.
- Una demostración totalmente formal (por ejemplo, escrita en el lenguaje de programación Lean 4) convence a una computadora, pero es ilegible para un par humano.
- Lo que se considera una demostración correcta depende del **contexto**: qué podemos asumir del lector, y para qué estamos demostrando.

¿A quién queremos convencer?

- Una demostración *heurística* (« $n! + 1$ no es divisible por nadie debajo de n , luego es primo») puede convencer a un apurado... y ser falsa: $4! + 1 = 25 = 5^2$.
- Una demostración totalmente formal (por ejemplo, escrita en el lenguaje de programación Lean 4) convence a una computadora, pero es ilegible para un par humano.
- Lo que se considera una demostración correcta depende del **contexto**: qué podemos asumir del lector, y para qué estamos demostrando.

Definición

Una **demostración** es un argumento formal sobre la veracidad de una proposición, que puede convencer a cualquier *par* de la comunidad científica.

El doble propósito de sus demostraciones

En el contexto de la materia, sus demostraciones tienen que:

El doble propósito de sus demostraciones

En el contexto de la materia, sus demostraciones tienen que:

- 1 **Convencer al lector** de la veracidad de la proposición. Esto es común a todas las demostraciones matemáticas.

El doble propósito de sus demostraciones

En el contexto de la materia, sus demostraciones tienen que:

- 1 **Convencer al lector** de la veracidad de la proposición. Esto es común a todas las demostraciones matemáticas.
- 2 **Convencer al docente de que entienden cómo convencer a cualquiera.** El docente ya sabe que la proposición es cierta. Va a evaluar si sus argumentos convencerían a *cualquier* par.

El doble propósito de sus demostraciones

En el contexto de la materia, sus demostraciones tienen que:

- 1 **Convencer al lector** de la veracidad de la proposición. Esto es común a todas las demostraciones matemáticas.
 - 2 **Convencer al docente de que entienden cómo convencer a cualquiera.** El docente ya sabe que la proposición es cierta. Va a evaluar si sus argumentos convencerían a *cualquier* par.
- Hay que ser un poco *paranoicos*: que no quede ninguna duda en la mente de ningún par que nos lea.

El doble propósito de sus demostraciones

En el contexto de la materia, sus demostraciones tienen que:

- ➊ **Convencer al lector** de la veracidad de la proposición. Esto es común a todas las demostraciones matemáticas.
 - ➋ **Convencer al docente de que entienden cómo convencer a cualquiera.** El docente ya sabe que la proposición es cierta. Va a evaluar si sus argumentos convencerían a *cualquier* par.
- Hay que ser un poco *paranoicos*: que no quede ninguna duda en la mente de ningún par que nos lea.
 - Desarrollar un pensamiento crítico-adversarial.

Formalidad y rigor

- La **formalidad** se refiere a la *forma* en la que escribimos. Las demostraciones existen en un continuo de formalismo: desde argumentos heurísticos hasta demostraciones verificables por computadora.

Formalidad y rigor

- La **formalidad** se refiere a la *forma* en la que escribimos. Las demostraciones existen en un continuo de formalismo: desde argumentos heurísticos hasta demostraciones verificables por computadora.
- El **rigor** se refiere a la *implicación lógica* de nuestras oraciones: el lector debería poder seguir la demostración paso a paso, sin preguntarse «¿y esto por qué vale?» a cada momento.

Formalidad y rigor

- La **formalidad** se refiere a la *forma* en la que escribimos. Las demostraciones existen en un continuo de formalismo: desde argumentos heurísticos hasta demostraciones verificables por computadora.
- El **rigor** se refiere a la *implicación lógica* de nuestras oraciones: el lector debería poder seguir la demostración paso a paso, sin preguntarse «¿y esto por qué vale?» a cada momento.

Importante

Se espera que puedan escribir y leer demostraciones entre los niveles «razonablemente formal» y «obviamente formal». La formalidad se usa para no cometer errores mientras aprenden. Debería ayudarlos a identificar lagunas en sus razonamientos.

Niveles de formalismo: un ejemplo

Proposición: un conjunto de n elementos tiene exactamente 2^n subconjuntos.

Niveles de formalismo: un ejemplo

Proposición: un conjunto de n elementos tiene exactamente 2^n subconjuntos.

Heurístico

Cada cosa puede o estar o no estar, así que hay $2 \cdot 2 \cdots 2 = 2^n$ subconjuntos.

Niveles de formalismo: un ejemplo

Proposición: un conjunto de n elementos tiene exactamente 2^n subconjuntos.

Heurístico

Cada cosa puede o estar o no estar, así que hay $2 \cdot 2 \cdots 2 = 2^n$ subconjuntos.

Razonablemente formal

Sea X un conjunto con $|X| = n$. Todo subconjunto $A \subseteq X$ queda determinado por su función indicadora $f_A : X \rightarrow \{0, 1\}$, donde $f_A(x) = 1$ si y sólo si $x \in A$. La correspondencia $A \mapsto f_A$ es una biyección entre $\mathcal{P}(X)$ y las funciones de X en $\{0, 1\}$, y hay exactamente $2^{|X|} = 2^n$ de tales funciones. Luego $|\mathcal{P}(X)| = 2^n$.

Niveles de formalismo: un ejemplo

Proposición: un conjunto de n elementos tiene exactamente 2^n subconjuntos.

Heurístico

Cada cosa puede o estar o no estar, así que hay $2 \cdot 2 \cdots 2 = 2^n$ subconjuntos.

Razonablemente formal

Sea X un conjunto con $|X| = n$. Todo subconjunto $A \subseteq X$ queda determinado por su función indicadora $f_A : X \rightarrow \{0, 1\}$, donde $f_A(x) = 1$ si y sólo si $x \in A$. La correspondencia $A \mapsto f_A$ es una biyección entre $\mathcal{P}(X)$ y las funciones de X en $\{0, 1\}$, y hay exactamente $2^{|X|} = 2^n$ de tales funciones. Luego $|\mathcal{P}(X)| = 2^n$.

La segunda nombra los objetos, explicita las relaciones, y cada oración se sigue de la anterior.

¿Los convence esta demostración?

Ejercicio: calcular la complejidad de un algoritmo que utiliza $T(n)$ pasos para una entrada de tamaño n , donde $T(n) = 2 T(n - 4)$.

Demostración de un alumno (verbatim)

$$T(n) = 2 T(n - 4) = 2(2 T(n - 4 - 4)) = \dots = 2^i T(n - 4i)$$

Como $n - 4i = 1 \Leftrightarrow i = \frac{n-1}{4}$. Luego,

$$= 2^{\frac{n-1}{4}} T(1) = (2^{\frac{1}{4}})^n 2^{-\frac{1}{4}} = O((2^{\frac{1}{4}})^n) \quad \square$$

¿Los convence esta demostración?

Ejercicio: calcular la complejidad de un algoritmo que utiliza $T(n)$ pasos para una entrada de tamaño n , donde $T(n) = 2 T(n - 4)$.

Demostración de un alumno (verbatim)

$$T(n) = 2 T(n - 4) = 2(2 T(n - 4 - 4)) = \dots = 2^i T(n - 4i)$$

Como $n - 4i = 1 \Leftrightarrow i = \frac{n-1}{4}$. Luego,

$$= 2^{\frac{n-1}{4}} T(1) = (2^{\frac{1}{4}})^n 2^{-\frac{1}{4}} = O((2^{\frac{1}{4}})^n) \quad \square$$

¿Los convence?

¿Los convence esta demostración?

Ejercicio: calcular la complejidad de un algoritmo que utiliza $T(n)$ pasos para una entrada de tamaño n , donde $T(n) = 2 T(n - 4)$.

Demostración de un alumno (verbatim)

$$T(n) = 2 T(n - 4) = 2(2 T(n - 4 - 4)) = \dots = 2^i T(n - 4i)$$

Como $n - 4i = 1 \Leftrightarrow i = \frac{n-1}{4}$. Luego,

$$= 2^{\frac{n-1}{4}} T(1) = (2^{\frac{1}{4}})^n 2^{-\frac{1}{4}} = O((2^{\frac{1}{4}})^n) \quad \square$$

¿Los convence? ¿Y si n no es congruente con 1 módulo 4?

¿Los convence esta demostración?

Ejercicio: calcular la complejidad de un algoritmo que utiliza $T(n)$ pasos para una entrada de tamaño n , donde $T(n) = 2 T(n - 4)$.

Demostración de un alumno (verbatim)

$$T(n) = 2 T(n - 4) = 2(2 T(n - 4 - 4)) = \dots = 2^i T(n - 4i)$$

Como $n - 4i = 1 \Leftrightarrow i = \frac{n-1}{4}$. Luego,

$$= 2^{\frac{n-1}{4}} T(1) = (2^{\frac{1}{4}})^n 2^{-\frac{1}{4}} = O((2^{\frac{1}{4}})^n) \quad \square$$

¿Los convence? ¿Y si n no es congruente con 1 módulo 4? ¿Cómo sabemos que $T(1) = 1$?

¿Los convence esta demostración?

Ejercicio: calcular la complejidad de un algoritmo que utiliza $T(n)$ pasos para una entrada de tamaño n , donde $T(n) = 2 T(n - 4)$.

Demostración de un alumno (verbatim)

$$T(n) = 2 T(n - 4) = 2(2 T(n - 4 - 4)) = \dots = 2^i T(n - 4i)$$

Como $n - 4i = 1 \Leftrightarrow i = \frac{n-1}{4}$. Luego,

$$= 2^{\frac{n-1}{4}} T(1) = (2^{\frac{1}{4}})^n 2^{-\frac{1}{4}} = O((2^{\frac{1}{4}})^n) \quad \square$$

¿Los convence? ¿Y si n no es congruente con 1 módulo 4? ¿Cómo sabemos que $T(1) = 1$?

El resultado es *cierto*, pero la demostración no muestra que el alumno entiende inducción, funciones recursivas ni comportamiento asintótico. **Hoy vamos a ver cómo hacerla bien.**

¿Cómo demostramos?

Al escribir una demostración, en general vamos a seguir estos pasos:

¿Cómo demostramos?

Al escribir una demostración, en general vamos a seguir estos pasos:

- 1 **Formalizar la consigna.** Traducir el enunciado a objetos matemáticos.

¿Cómo demostramos?

Al escribir una demostración, en general vamos a seguir estos pasos:

- 1 **Formalizar la consigna.** Traducir el enunciado a objetos matemáticos.
- 2 **Comprender qué se nos pide.** ¿Qué asumimos? ¿Qué hay que probar?

¿Cómo demostramos?

Al escribir una demostración, en general vamos a seguir estos pasos:

- 1 **Formalizar la consigna.** Traducir el enunciado a objetos matemáticos.
- 2 **Comprender qué se nos pide.** ¿Qué asumimos? ¿Qué hay que probar?
- 3 **Considerar ejemplos.** Jugar con casos chicos, buscar contraejemplos.

¿Cómo demostramos?

Al escribir una demostración, en general vamos a seguir estos pasos:

- 1 **Formalizar la consigna.** Traducir el enunciado a objetos matemáticos.
- 2 **Comprender qué se nos pide.** ¿Qué asumimos? ¿Qué hay que probar?
- 3 **Considerar ejemplos.** Jugar con casos chicos, buscar contraejemplos.
- 4 **Encontrar un argumento intuitivo.** ¿Por qué el resultado es cierto?

¿Cómo demostramos?

Al escribir una demostración, en general vamos a seguir estos pasos:

- 1 **Formalizar la consigna.** Traducir el enunciado a objetos matemáticos.
- 2 **Comprender qué se nos pide.** ¿Qué asumimos? ¿Qué hay que probar?
- 3 **Considerar ejemplos.** Jugar con casos chicos, buscar contraejemplos.
- 4 **Encontrar un argumento intuitivo.** ¿Por qué el resultado es cierto?
- 5 **Elegir una estrategia.** Inducción, contradicción, reducción al absurdo, partir en casos, combinación de estrategias, etc.

¿Cómo demostramos?

Al escribir una demostración, en general vamos a seguir estos pasos:

- 1 **Formalizar la consigna.** Traducir el enunciado a objetos matemáticos.
- 2 **Comprender qué se nos pide.** ¿Qué asumimos? ¿Qué hay que probar?
- 3 **Considerar ejemplos.** Jugar con casos chicos, buscar contraejemplos.
- 4 **Encontrar un argumento intuitivo.** ¿Por qué el resultado es cierto?
- 5 **Elegir una estrategia.** Inducción, contradicción, reducción al absurdo, partir en casos, combinación de estrategias, etc.
- 6 **Pasar en limpio.** Que el lector pueda seguir cada paso.

Formalizar la consigna

Rara vez nos dan el problema pre-formalizado. **Si formalizamos mal, todo lo que hagamos después es irrelevante.**

Formalizar la consigna

Rara vez nos dan el problema pre-formalizado. **Si formalizamos mal, todo lo que hagamos después es irrelevante.**

Ejercicio

Una colonia de bacterias se triplica cada hora. Si al comenzar hay 5 bacterias, ¿cuántas hay después de n horas? Probar la respuesta.

Formalizar la consigna

Rara vez nos dan el problema pre-formalizado. **Si formalizamos mal, todo lo que hagamos después es irrelevante.**

Ejercicio

Una colonia de bacterias se triplica cada hora. Si al comenzar hay 5 bacterias, ¿cuántas hay después de n horas? Probar la respuesta.

- Habla de un proceso que se repite: una **sucesión definida por recurrencia** parece un buen modelo.

Formalizar la consigna

Rara vez nos dan el problema pre-formalizado. **Si formalizamos mal, todo lo que hagamos después es irrelevante.**

Ejercicio

Una colonia de bacterias se triplica cada hora. Si al comenzar hay 5 bacterias, ¿cuántas hay después de n horas? Probar la respuesta.

- Habla de un proceso que se repite: una **sucesión definida por recurrencia** parece un buen modelo.
- ¿Se triplica *exactamente*? ¿Ninguna muere? ¿«Después de n horas» cuenta desde la hora 0 o desde la 1? Si el enunciado no lo aclara, ¡preguntar!

Formalizar la consigna

Rara vez nos dan el problema pre-formalizado. **Si formalizamos mal, todo lo que hagamos después es irrelevante.**

Ejercicio

Una colonia de bacterias se triplica cada hora. Si al comenzar hay 5 bacterias, ¿cuántas hay después de n horas? Probar la respuesta.

- Habla de un proceso que se repite: una **sucesión definida por recurrencia** parece un buen modelo.
- ¿Se triplica *exactamente*? ¿Ninguna muere? ¿«Después de n horas» cuenta desde la hora 0 o desde la 1? Si el enunciado no lo aclara, ¡preguntar!

Versión formalizada

Sea $b_0 = 5$, y $b_{n+1} = 3 \cdot b_n$ para todo $n \in \mathbb{N}$. Probar que: $\forall n \in \mathbb{N}, b_n = 5 \cdot 3^n$.

¿Qué ganamos al formalizar?

La versión formal:

¿Qué ganamos al formalizar?

La versión formal:

- **Nombra** los objetos de los que habla (la sucesión b , la cantidad b_n , la hora n).

¿Qué ganamos al formalizar?

La versión formal:

- **Nombra** los objetos de los que habla (la sucesión b , la cantidad b_n , la hora n).
- **Explicita relaciones** formalmente sobre los mismos ($b_0 = 5$, $b_{n+1} = 3 \cdot b_n$, $b_n = 5 \cdot 3^n \forall n \in \mathbb{N}$).

¿Qué ganamos al formalizar?

La versión formal:

- **Nombra** los objetos de los que habla (la sucesión b , la cantidad b_n , la hora n).
- **Explicita relaciones** formalmente sobre los mismos ($b_0 = 5$, $b_{n+1} = 3 \cdot b_n$, $b_n = 5 \cdot 3^n \forall n \in \mathbb{N}$).
- **Cuantifica** las variables usadas («sea», «para todo», «existe»).

¿Qué ganamos al formalizar?

La versión formal:

- **Nombra** los objetos de los que habla (la sucesión b , la cantidad b_n , la hora n).
- **Explicita relaciones** formalmente sobre los mismos ($b_0 = 5$, $b_{n+1} = 3 \cdot b_n$, $b_n = 5 \cdot 3^n \forall n \in \mathbb{N}$).
- **Cuantifica** las variables usadas («sea», «para todo», «existe»).
- Usa **conectores lógicos** («si», «entonces», «luego», «porque»).

¿Qué ganamos al formalizar?

La versión formal:

- **Nombra** los objetos de los que habla (la sucesión b , la cantidad b_n , la hora n).
- **Explicita relaciones** formalmente sobre los mismos ($b_0 = 5$, $b_{n+1} = 3 \cdot b_n$, $b_n = 5 \cdot 3^n \forall n \in \mathbb{N}$).
- **Cuantifica** las variables usadas («sea», «para todo», «existe»).
- Usa **conectores lógicos** («si», «entonces», «luego», «porque»).

Comparar con: «*después de un rato hay el triple del triple del triple. . . de 5*». ¿Cuántas veces «el triple»? ¿ n o $n + 1$? Distintas personas lo interpretan de distintas maneras.

Comprender qué se nos pide: la conversación

Una herramienta útil: pensar la demostración como una **conversación** entre quien demuestra (Alicia) y un escéptico (Beto).

Comprender qué se nos pide: la conversación

Una herramienta útil: pensar la demostración como una **conversación** entre quien demuestra (Alicia) y un escéptico (Beto).

- Probar $\forall x.P(x)$: Beto elige el x que quiere, Alicia tiene que responder para ese x . No podemos elegir nosotros el caso cómodo.

Comprender qué se nos pide: la conversación

Una herramienta útil: pensar la demostración como una **conversación** entre quien demuestra (Alicia) y un escéptico (Beto).

- Probar $\forall x.P(x)$: Beto elige el x que quiere, Alicia tiene que responder para ese x . No podemos elegir nosotros el caso cómodo.
- Probar $\exists x.P(x)$: Alicia da un x concreto, y muestra que cumple P .

Comprender qué se nos pide: la conversación

Una herramienta útil: pensar la demostración como una **conversación** entre quien demuestra (Alicia) y un escéptico (Beto).

- Probar $\forall x.P(x)$: Beto elige el x que quiere, Alicia tiene que responder para ese x . No podemos elegir nosotros el caso cómodo.
- Probar $\exists x.P(x)$: Alicia da un x concreto, y muestra que cumple P .
- Probar $P \Rightarrow Q$: asumimos P (¡y lo decimos!), y deducimos Q .

Comprender qué se nos pide: la conversación

Una herramienta útil: pensar la demostración como una **conversación** entre quien demuestra (Alicia) y un escéptico (Beto).

- Probar $\forall x.P(x)$: Beto elige el x que quiere, Alicia tiene que responder para ese x . No podemos elegir nosotros el caso cómodo.
- Probar $\exists x.P(x)$: Alicia da un x concreto, y muestra que cumple P .
- Probar $P \Rightarrow Q$: asumimos P (¡y lo decimos!), y deducimos Q .
- En $\forall x.\exists y.P(x, y)$: el y puede depender del x . En $\exists y.\forall x.P(x, y)$, no. Son proposiciones completamente distintas.

Comprender qué se nos pide: la conversación

Una herramienta útil: pensar la demostración como una **conversación** entre quien demuestra (Alicia) y un escéptico (Beto).

- Probar $\forall x.P(x)$: Beto elige el x que quiere, Alicia tiene que responder para ese x . No podemos elegir nosotros el caso cómodo.
- Probar $\exists x.P(x)$: Alicia da un x concreto, y muestra que cumple P .
- Probar $P \Rightarrow Q$: asumimos P (¡y lo decimos!), y deducimos Q .
- En $\forall x.\exists y.P(x, y)$: el y puede depender del x . En $\exists y.\forall x.P(x, y)$, no. Son proposiciones completamente distintas.

Ejemplo

«Para todo $\varepsilon > 0$ existe $\delta > 0$ tal que...»: Beto da $\varepsilon = 0,2$ y desafía; Alicia responde $\delta = 0,4$ y justifica. Nuestra demostración tiene que ganar esa conversación *para cualquier jugada de Beto*.

Considerar ejemplos (pero no confundirlos con demostraciones)

- Probar casos chicos nos ayuda a **entender** el problema, conjeturar la respuesta, y detectar errores en la consigna o en nuestra intuición.

Considerar ejemplos (pero no confundirlos con demostraciones)

- Probar casos chicos nos ayuda a **entender** el problema, conjeturar la respuesta, y detectar errores en la consigna o en nuestra intuición.
- Buscar **contraejemplos** nos dice qué hipótesis son necesarias.

Considerar ejemplos (pero no confundirlos con demostraciones)

- Probar casos chicos nos ayuda a **entender** el problema, conjeturar la respuesta, y detectar errores en la consigna o en nuestra intuición.
- Buscar **contraejemplos** nos dice qué hipótesis son necesarias.

Pero los ejemplos no demuestran

Conjetura (Goldbach). Para todo $n \in \mathbb{N}$ par, $n > 2$, existen primos p, q tales que
$$n = p + q.$$

Todos los n pares mayores que 2 que se verificaron hasta el momento cumplen la afirmación... y jamás fue demostrada, después de cientos de años de intentos.

Considerar ejemplos (pero no confundirlos con demostraciones)

- Probar casos chicos nos ayuda a **entender** el problema, conjeturar la respuesta, y detectar errores en la consigna o en nuestra intuición.
- Buscar **contraejemplos** nos dice qué hipótesis son necesarias.

Pero los ejemplos no demuestran

Conjetura (Goldbach). Para todo $n \in \mathbb{N}$ par, $n > 2$, existen primos p, q tales que
$$n = p + q.$$

Todos los n pares mayores que 2 que se verificaron hasta el momento cumplen la afirmación... y jamás fue demostrada, después de cientos de años de intentos.

«Verificar que es cierto para todos los casos que se me ocurren» no es una demostración.

Repaso: complejidad asintótica

Comparamos funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ «para n grande, salvo constantes».

Repaso: complejidad asintótica

Comparamos funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ «para n grande, salvo constantes».

Cota superior

$g(n) \in O(f(n))$ si existe una constante positiva c y entero no negativo n_0 tales que

$$g(n) \leq c \cdot f(n) \quad \text{para todo } n \geq n_0.$$

Repaso: complejidad asintótica

Comparamos funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ «para n grande, salvo constantes».

Cota superior

$g(n) \in O(f(n))$ si existe una constante positiva c y entero no negativo n_0 tales que

$$g(n) \leq c \cdot f(n) \quad \text{para todo } n \geq n_0.$$

Cota inferior

$g(n) \in \Omega(f(n))$ si existe una constante positiva c y entero no negativo n_0 tales que

$$g(n) \geq c \cdot f(n) \quad \text{para todo } n \geq n_0.$$

Repaso: complejidad asintótica

Comparamos funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ «para n grande, salvo constantes».

Cota superior

$g(n) \in O(f(n))$ si existe una constante positiva c y entero no negativo n_0 tales que

$$g(n) \leq c \cdot f(n) \quad \text{para todo } n \geq n_0.$$

Cota inferior

$g(n) \in \Omega(f(n))$ si existe una constante positiva c y entero no negativo n_0 tales que

$$g(n) \geq c \cdot f(n) \quad \text{para todo } n \geq n_0.$$

Orden exacto

$g(n) \in \Theta(f(n))$ si $g(n) \in O(f(n))$ y $g(n) \in \Omega(f(n))$.

¿Cómo se demuestra una pertenencia a O ?

Para probar $g(n) \in O(f(n))$ hay que **exhibir** las constantes c y n_0 , y probar la desigualdad para todo $n \geq n_0$. Es un $\exists$: nosotros elegimos.

¿Cómo se demuestra una pertenencia a O ?

Para probar $g(n) \in O(f(n))$ hay que **exhibir** las constantes c y n_0 , y probar la desigualdad para todo $n \geq n_0$. Es un $\exists$: nosotros elegimos.

Ejemplo

Probar que $3n^2 + 10n \in O(n^2)$.

¿Cómo se demuestra una pertenencia a O ?

Para probar $g(n) \in O(f(n))$ hay que **exhibir** las constantes c y n_0 , y probar la desigualdad para todo $n \geq n_0$. Es un $\exists$: nosotros elegimos.

Ejemplo

Probar que $3n^2 + 10n \in O(n^2)$.

Demostración. Elegimos $c = 13$ y $n_0 = 1$. Sea $n \geq 1$. Como $n \geq 1$, vale $n \leq n^2$, entonces

$$3n^2 + 10n \leq 3n^2 + 10n^2 = 13n^2 = c \cdot n^2.$$

Luego, para todo $n \geq n_0$, $3n^2 + 10n \leq c \cdot n^2$, que es lo que queríamos demostrar. $\square$

¿Cómo se demuestra una pertenencia a O ?

Para probar $g(n) \in O(f(n))$ hay que **exhibir** las constantes c y n_0 , y probar la desigualdad para todo $n \geq n_0$. Es un $\exists$: nosotros elegimos.

Ejemplo

Probar que $3n^2 + 10n \in O(n^2)$.

Demostración. Elegimos $c = 13$ y $n_0 = 1$. Sea $n \geq 1$. Como $n \geq 1$, vale $n \leq n^2$, entonces

$$3n^2 + 10n \leq 3n^2 + 10n^2 = 13n^2 = c \cdot n^2.$$

Luego, para todo $n \geq n_0$, $3n^2 + 10n \leq c \cdot n^2$, que es lo que queríamos demostrar. $\square$

- Las constantes no tienen que ser ajustadas: $c = 1000$, $n_0 = 40$ también sirven.

¿Cómo se demuestra una pertenencia a O ?

Para probar $g(n) \in O(f(n))$ hay que **exhibir** las constantes c y n_0 , y probar la desigualdad para todo $n \geq n_0$. Es un $\exists$: nosotros elegimos.

Ejemplo

Probar que $3n^2 + 10n \in O(n^2)$.

Demostración. Elegimos $c = 13$ y $n_0 = 1$. Sea $n \geq 1$. Como $n \geq 1$, vale $n \leq n^2$, entonces

$$3n^2 + 10n \leq 3n^2 + 10n^2 = 13n^2 = c \cdot n^2.$$

Luego, para todo $n \geq n_0$, $3n^2 + 10n \leq c \cdot n^2$, que es lo que queríamos demostrar. $\square$

- Las constantes no tienen que ser ajustadas: $c = 1000$, $n_0 = 40$ también sirven.
- Noten que *dijimos* dónde usamos $n \geq 1$.

¿Y para probar que *no* pertenece?

Para probar $g(n) \notin O(f(n))$ tenemos que **negar** un $\exists$: para *toda* elección de c y n_0 , existe un $n \geq n_0$ que rompe la desigualdad.

¿Y para probar que *no* pertenece?

Para probar $g(n) \notin O(f(n))$ tenemos que **negar** un $\exists$: para *toda* elección de c y n_0 , existe un $n \geq n_0$ que rompe la desigualdad.

Ejemplo

Probar que $n^2 \notin O(n)$.

¿Y para probar que *no* pertenece?

Para probar $g(n) \notin O(f(n))$ tenemos que **negar** un $\exists$: para *toda* elección de c y n_0 , existe un $n \geq n_0$ que rompe la desigualdad.

Ejemplo

Probar que $n^2 \notin O(n)$.

Demostración. Sean c y n_0 constantes positivas cualesquiera. Tomemos $n = \max(n_0, \lceil c \rceil + 1)$. Entonces $n \geq n_0$, y como $n > c$, multiplicando por $n > 0$ a ambos lados obtenemos $n^2 > c \cdot n$. Luego no se cumple $n^2 \leq c \cdot n$ para todo $n \geq n_0$. Como c y n_0 eran arbitrarias, ninguna elección de constantes sirve, y $n^2 \notin O(n)$. $\square$

¿Y para probar que *no* pertenece?

Para probar $g(n) \notin O(f(n))$ tenemos que **negar** un $\exists$: para *toda* elección de c y n_0 , existe un $n \geq n_0$ que rompe la desigualdad.

Ejemplo

Probar que $n^2 \notin O(n)$.

Demostración. Sean c y n_0 constantes positivas cualesquiera. Tomemos $n = \max(n_0, \lceil c \rceil + 1)$. Entonces $n \geq n_0$, y como $n > c$, multiplicando por $n > 0$ a ambos lados obtenemos $n^2 > c \cdot n$. Luego no se cumple $n^2 \leq c \cdot n$ para todo $n \geq n_0$. Como c y n_0 eran arbitrarias, ninguna elección de constantes sirve, y $n^2 \notin O(n)$. $\square$

Otra vez la conversación: Beto nos da c y n_0 , y nosotros respondemos con un n que depende de ellas.

Propiedades útiles (para no sufrir)

- **Transitividad:** si $f \in O(g)$ y $g \in O(h)$, entonces $f \in O(h)$.

Propiedades útiles (para no sufrir)

- **Transitividad:** si $f \in O(g)$ y $g \in O(h)$, entonces $f \in O(h)$.
- **Suma:** si $f_1 \in O(g)$ y $f_2 \in O(g)$, entonces $f_1 + f_2 \in O(g)$.

Propiedades útiles (para no sufrir)

- **Transitividad:** si $f \in O(g)$ y $g \in O(h)$, entonces $f \in O(h)$.
- **Suma:** si $f_1 \in O(g)$ y $f_2 \in O(g)$, entonces $f_1 + f_2 \in O(g)$.
- **Simetría:** $g \in O(f) \iff f \in \Omega(g)$. (¡Sale directo de las definiciones!)

Propiedades útiles (para no sufrir)

- **Transitividad:** si $f \in O(g)$ y $g \in O(h)$, entonces $f \in O(h)$.
- **Suma:** si $f_1 \in O(g)$ y $f_2 \in O(g)$, entonces $f_1 + f_2 \in O(g)$.
- **Simetría:** $g \in O(f) \iff f \in \Omega(g)$. (¡Sale directo de las definiciones!)
- Los polinomios están dominados por su término de mayor grado, y 2^n crece más que cualquier polinomio ($n!$ crece más que 2^n).

El principio de inducción

Sea $P(n)$ una proposición sobre los números naturales. Si probamos:

El principio de inducción

Sea $P(n)$ una proposición sobre los números naturales. Si probamos:

- 1 **Caso base:** $P(0)$ es cierta (podría ser otro).

El principio de inducción

Sea $P(n)$ una proposición sobre los números naturales. Si probamos:

- 1 **Caso base:** $P(0)$ es cierta (podría ser otro).
- 2 **Paso inductivo:** para todo $n \in \mathbb{N}$, $P(n) \Rightarrow P(n + 1)$.

El principio de inducción

Sea $P(n)$ una proposición sobre los números naturales. Si probamos:

- 1 **Caso base:** $P(0)$ es cierta (podría ser otro).
- 2 **Paso inductivo:** para todo $n \in \mathbb{N}$, $P(n) \Rightarrow P(n+1)$.

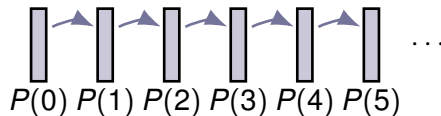
entonces $P(n)$ es cierta **para todo** $n \in \mathbb{N}$.

El principio de inducción

Sea $P(n)$ una proposición sobre los números naturales. Si probamos:

- 1 **Caso base:** $P(0)$ es cierta (podría ser otro).
- 2 **Paso inductivo:** para todo $n \in \mathbb{N}$, $P(n) \Rightarrow P(n+1)$.

entonces $P(n)$ es cierta **para todo** $n \in \mathbb{N}$.

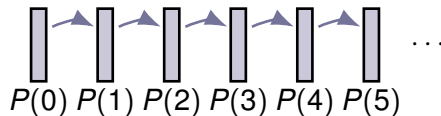


El principio de inducción

Sea $P(n)$ una proposición sobre los números naturales. Si probamos:

- 1 **Caso base:** $P(0)$ es cierta (podría ser otro).
- 2 **Paso inductivo:** para todo $n \in \mathbb{N}$, $P(n) \Rightarrow P(n+1)$.

entonces $P(n)$ es cierta **para todo** $n \in \mathbb{N}$.



Como el dominó: si cae la primera ficha, y cada ficha tira la siguiente, caen todas.

Cómo se escribe una demostración por inducción

- 1 **Definir explícitamente** $P(n)$, la proposición sobre los naturales, con sus cuantificadores. Este es el paso que más se saltan... y donde nacen casi todos los errores.

Cómo se escribe una demostración por inducción

- 1 **Definir explícitamente** $P(n)$, la proposición sobre los naturales, con sus cuantificadores. Este es el paso que más se saltean... y donde nacen casi todos los errores.
- 2 **Probar el caso base** (¡o los casos base, ya vamos a ver!).

Cómo se escribe una demostración por inducción

- 1 **Definir explícitamente** $P(n)$, la proposición sobre los naturales, con sus cuantificadores. Este es el paso que más se saltan... y donde nacen casi todos los errores.
- 2 **Probar el caso base** (¡o los casos base, ya vamos a ver!).
- 3 **Probar el paso inductivo:** asumir la hipótesis inductiva (HI), *decir que la asumimos*, y deducir $P(n + 1)$, marcando *dónde* usamos la HI.

Cómo se escribe una demostración por inducción

- 1 **Definir explícitamente** $P(n)$, la proposición sobre los naturales, con sus cuantificadores. Este es el paso que más se saltan... y donde nacen casi todos los errores.
- 2 **Probar el caso base** (¡o los casos base, ya vamos a ver!).
- 3 **Probar el paso inductivo:** asumir la hipótesis inductiva (HI), *decir que la asumimos*, y deducir $P(n+1)$, marcando *dónde* usamos la HI.
- 4 **Concluir:** «por inducción, $P(n)$ vale para todo $n \in \mathbb{N}$ ».

Cómo se escribe una demostración por inducción

- 1 **Definir explícitamente** $P(n)$, la proposición sobre los naturales, con sus cuantificadores. Este es el paso que más se saltan... y donde nacen casi todos los errores.
- 2 **Probar el caso base** (¡o los casos base, ya vamos a ver!).
- 3 **Probar el paso inductivo:** asumir la hipótesis inductiva (HI), *decir que la asumimos*, y deducir $P(n+1)$, marcando *dónde* usamos la HI.
- 4 **Concluir:** «por inducción, $P(n)$ vale para todo $n \in \mathbb{N}$ ».

Advertencia

La inducción es **sobre naturales**. No es «sobre conjuntos», ni «sobre secuencias». Si quieren hacer inducción sobre otra estructura, la propiedad P tiene que hablar de un *tamaño* natural de esa estructura.

Ejemplo: una suma geométrica

Proposición

Para todo $n \in \mathbb{N}$, $\sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}$.

Ejemplo: una suma geométrica

Proposición

Para todo $n \in \mathbb{N}$,
$$\sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$$

El plan, antes de escribir:

Ejemplo: una suma geométrica

Proposición

Para todo $n \in \mathbb{N}$,
$$\sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$$

El plan, antes de escribir:

- **La propiedad:** $P(n) : \sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$

Ejemplo: una suma geométrica

Proposición

Para todo $n \in \mathbb{N}$,
$$\sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$$

El plan, antes de escribir:

- **La propiedad:** $P(n) : \sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$
- **Caso base:** $P(0)$. Con uno alcanza: el paso inductivo sólo va a usar $P(n)$.

Ejemplo: una suma geométrica

Proposición

Para todo $n \in \mathbb{N}$,
$$\sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$$

El plan, antes de escribir:

- **La propiedad:** $P(n) : \sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$
- **Caso base:** $P(0)$. Con uno alcanza: el paso inductivo sólo va a usar $P(n)$.
- **Paso inductivo:** separar el último término, $\sum_{i=0}^{n+1} 3^i = (\sum_{i=0}^n 3^i) + 3^{n+1}$: ahí entra la HI.

Ejemplo: una suma geométrica

Proposición

Para todo $n \in \mathbb{N}$,
$$\sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$$

El plan, antes de escribir:

- **La propiedad:** $P(n) : \sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$
- **Caso base:** $P(0)$. Con uno alcanza: el paso inductivo sólo va a usar $P(n)$.
- **Paso inductivo:** separar el último término, $\sum_{i=0}^{n+1} 3^i = (\sum_{i=0}^n 3^i) + 3^{n+1}$: ahí entra la HI.

→ Demostración completa en el pizarrón.

Ejemplo: una suma geométrica

Proposición

Para todo $n \in \mathbb{N}$,
$$\sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$$

El plan, antes de escribir:

- **La propiedad:** $P(n) : \sum_{i=0}^n 3^i = \frac{3^{n+1} - 1}{2}.$
- **Caso base:** $P(0)$. Con uno alcanza: el paso inductivo sólo va a usar $P(n)$.
- **Paso inductivo:** separar el último término, $\sum_{i=0}^{n+1} 3^i = (\sum_{i=0}^n 3^i) + 3^{n+1}$: ahí entra la HI.

→ Demostración completa en el pizarrón.

Revisen si: definimos P , probamos el caso base y *dijimos* dónde usamos la HI.

El caso base no tiene por qué ser 0

A veces la propiedad vale recién a partir de un $n_0 > 0$. La inducción funciona igual, empezando el caso base en n_0 .

El caso base no tiene por qué ser 0

A veces la propiedad vale recién a partir de un $n_0 > 0$. La inducción funciona igual, empezando el caso base en n_0 .

Proposición

Para todo $n \geq 4$, $n! > 2^n$.

El caso base no tiene por qué ser 0

A veces la propiedad vale recién a partir de un $n_0 > 0$. La inducción funciona igual, empezando el caso base en n_0 .

Proposición

Para todo $n \geq 4$, $n! > 2^n$.

- ¿Y para $n < 4$? $0! = 1 = 2^0$, $1! = 1 < 2$, $2! = 2 < 4$, $3! = 6 < 8$: la propiedad es **falsa**. El caso base *corrido* no es un capricho.

El caso base no tiene por qué ser 0

A veces la propiedad vale recién a partir de un $n_0 > 0$. La inducción funciona igual, empezando el caso base en n_0 .

Proposición

Para todo $n \geq 4$, $n! > 2^n$.

- ¿Y para $n < 4$? $0! = 1 = 2^0$, $1! = 1 < 2$, $2! = 2 < 4$, $3! = 6 < 8$: la propiedad es **falsa**. El caso base *corrido* no es un capricho.
- El caso base pasa a ser $P(4)$, y en el paso inductivo tomamos $n \geq 4$: la HI la tenemos sólo a partir de ahí.

El caso base no tiene por qué ser 0

A veces la propiedad vale recién a partir de un $n_0 > 0$. La inducción funciona igual, empezando el caso base en n_0 .

Proposición

Para todo $n \geq 4$, $n! > 2^n$.

- ¿Y para $n < 4$? $0! = 1 = 2^0$, $1! = 1 < 2$, $2! = 2 < 4$, $3! = 6 < 8$: la propiedad es **falsa**. El caso base *corrido* no es un capricho.
- El caso base pasa a ser $P(4)$, y en el paso inductivo tomamos $n \geq 4$: la HI la tenemos sólo a partir de ahí.

→ Demostración en el pizarrón.

El caso base no tiene por qué ser 0

A veces la propiedad vale recién a partir de un $n_0 > 0$. La inducción funciona igual, empezando el caso base en n_0 .

Proposición

Para todo $n \geq 4$, $n! > 2^n$.

- ¿Y para $n < 4$? $0! = 1 = 2^0$, $1! = 1 < 2$, $2! = 2 < 4$, $3! = 6 < 8$: la propiedad es **falsa**. El caso base *corrido* no es un capricho.
- El caso base pasa a ser $P(4)$, y en el paso inductivo tomamos $n \geq 4$: la HI la tenemos sólo a partir de ahí.

→ Demostración en el pizarrón.

Pregunta para después del pizarrón: ¿en qué paso exacto se usa que $n \geq 4$?

Inducción fuerte (o global, o completa)

A veces para probar $P(n)$ no nos alcanza con $P(n - 1)$: necesitamos la propiedad para *varios* (o todos los) valores anteriores.

Inducción fuerte (o global, o completa)

A veces para probar $P(n)$ no nos alcanza con $P(n - 1)$: necesitamos la propiedad para *varios* (o todos los) valores anteriores.

Principio de inducción fuerte

Si para todo $n \in \mathbb{N}$ vale

$$\left(\forall k \in \mathbb{N}. k < n \Rightarrow P(k) \right) \Rightarrow P(n),$$

entonces $P(n)$ es cierta para todo $n \in \mathbb{N}$.

Inducción fuerte (o global, o completa)

A veces para probar $P(n)$ no nos alcanza con $P(n - 1)$: necesitamos la propiedad para *varios* (o todos los) valores anteriores.

Principio de inducción fuerte

Si para todo $n \in \mathbb{N}$ vale

$$\left(\forall k \in \mathbb{N}. k < n \Rightarrow P(k) \right) \Rightarrow P(n),$$

entonces $P(n)$ es cierta para todo $n \in \mathbb{N}$.

- La hipótesis inductiva ahora es: « P vale para **todos** los $k < n$ ».

Inducción fuerte (o global, o completa)

A veces para probar $P(n)$ no nos alcanza con $P(n-1)$: necesitamos la propiedad para *varios* (o todos los) valores anteriores.

Principio de inducción fuerte

Si para todo $n \in \mathbb{N}$ vale

$$\left(\forall k \in \mathbb{N}. k < n \Rightarrow P(k) \right) \Rightarrow P(n),$$

entonces $P(n)$ es cierta para todo $n \in \mathbb{N}$.

- La hipótesis inductiva ahora es: « P vale para **todos** los $k < n$ ».
- Es la herramienta natural cuando la recursión «salta»: $T(n)$ definido con $T(n-4)$, a_n definido con a_{n-1} y a_{n-2} , $\text{Exp}(a, n)$ que llama a $\text{Exp}(a, \lfloor n/2 \rfloor)$, ...

Inducción fuerte (o global, o completa)

A veces para probar $P(n)$ no nos alcanza con $P(n-1)$: necesitamos la propiedad para *varios* (o todos los) valores anteriores.

Principio de inducción fuerte

Si para todo $n \in \mathbb{N}$ vale

$$\left(\forall k \in \mathbb{N}. k < n \Rightarrow P(k) \right) \Rightarrow P(n),$$

entonces $P(n)$ es cierta para todo $n \in \mathbb{N}$.

- La hipótesis inductiva ahora es: « P vale para **todos** los $k < n$ ».
- Es la herramienta natural cuando la recursión «salta»: $T(n)$ definido con $T(n-4)$, a_n definido con a_{n-1} y a_{n-2} , $\text{Exp}(a, n)$ que llama a $\text{Exp}(a, \lfloor n/2 \rfloor)$, ...
- Equivalente a la inducción común, pero mucho más cómoda para recursiones.

¡Cuidado con cuántos casos base necesitamos!

Advertencia

Si nuestra demostración de $P(n)$ usa $P(n-1), P(n-2), \dots, P(n-k)$ para un $k \geq 1$ fijo, entonces necesitamos k **casos base**. Para $n < k$, « $P(n-k)$ » no tiene sentido: nos caemos de $\mathbb{N}$.

¡Cuidado con cuántos casos base necesitamos!

Advertencia

Si nuestra demostración de $P(n)$ usa $P(n-1), P(n-2), \dots, P(n-k)$ para un $k \geq 1$ fijo, entonces necesitamos k **casos base**. Para $n < k$, « $P(n-k)$ » no tiene sentido: nos caemos de $\mathbb{N}$.

- Si uso $P(n-1)$ y $P(n-2)$: pruebo a mano $P(0)$ y $P(1)$.

¡Cuidado con cuántos casos base necesitamos!

Advertencia

Si nuestra demostración de $P(n)$ usa $P(n-1), P(n-2), \dots, P(n-k)$ para un $k \geq 1$ fijo, entonces necesitamos k **casos base**. Para $n < k$, « $P(n-k)$ » no tiene sentido: nos caemos de $\mathbb{N}$.

- Si uso $P(n-1)$ y $P(n-2)$: pruebo a mano $P(0)$ y $P(1)$.
- Si uso $P(n-4)$: pruebo a mano $P(0), P(1), P(2), P(3)$.

¡Cuidado con cuántos casos base necesitamos!

Advertencia

Si nuestra demostración de $P(n)$ usa $P(n-1), P(n-2), \dots, P(n-k)$ para un $k \geq 1$ fijo, entonces necesitamos k **casos base**. Para $n < k$, « $P(n-k)$ » no tiene sentido: nos caemos de $\mathbb{N}$.

- Si uso $P(n-1)$ y $P(n-2)$: pruebo a mano $P(0)$ y $P(1)$.
- Si uso $P(n-4)$: pruebo a mano $P(0), P(1), P(2), P(3)$.
- Si uso $P(\lfloor n/2 \rfloor)$ con inducción fuerte: alcanza un solo caso base, $P(0)$, porque $\lfloor n/2 \rfloor < n$ para todo $n \geq 1$ y nunca nos caemos de $\mathbb{N}$.

¡Cuidado con cuántos casos base necesitamos!

Advertencia

Si nuestra demostración de $P(n)$ usa $P(n-1), P(n-2), \dots, P(n-k)$ para un $k \geq 1$ fijo, entonces necesitamos k **casos base**. Para $n < k$, « $P(n-k)$ » no tiene sentido: nos caemos de $\mathbb{N}$.

- Si uso $P(n-1)$ y $P(n-2)$: pruebo a mano $P(0)$ y $P(1)$.
- Si uso $P(n-4)$: pruebo a mano $P(0), P(1), P(2), P(3)$.
- Si uso $P(\lfloor n/2 \rfloor)$ con inducción fuerte: alcanza un solo caso base, $P(0)$, porque $\lfloor n/2 \rfloor < n$ para todo $n \geq 1$ y nunca nos caemos de $\mathbb{N}$.

Veamos el ejemplo del principio: la recurrencia $T(n) = 2 T(n-4)$, ahora con rigor.

Ahora sí: $T(n) = 2 T(n - 4)$, con rigor

El plan para hacerla bien:

Ahora sí: $T(n) = 2 T(n - 4)$, con rigor

El plan para hacerla bien:

- 1 **Definir T con su dominio:** sea $T : \mathbb{N} \rightarrow \mathbb{N}$ tal que $T(n) = 2 T(n - 4)$ para todo $n \geq 4$. De $T(0), \dots, T(3)$ no sabemos nada: sea $a = \max(T(0), T(1), T(2), T(3))$.

Ahora sí: $T(n) = 2 T(n - 4)$, con rigor

El plan para hacerla bien:

- 1 **Definir T con su dominio:** sea $T : \mathbb{N} \rightarrow \mathbb{N}$ tal que $T(n) = 2 T(n - 4)$ para todo $n \geq 4$. De $T(0), \dots, T(3)$ no sabemos nada: sea $a = \max(T(0), T(1), T(2), T(3))$.
- 2 **Definir la propiedad:** $P(n) : T(n) \leq a \cdot 2^{\frac{n}{4}}$.

Ahora sí: $T(n) = 2 T(n - 4)$, con rigor

El plan para hacerla bien:

- 1 **Definir T con su dominio:** sea $T : \mathbb{N} \rightarrow \mathbb{N}$ tal que $T(n) = 2 T(n - 4)$ para todo $n \geq 4$. De $T(0), \dots, T(3)$ no sabemos nada: sea $a = \max(T(0), T(1), T(2), T(3))$.
- 2 **Definir la propiedad:** $P(n) : T(n) \leq a \cdot 2^{\frac{n}{4}}$.
- 3 **Cuatro casos base** ($0 \leq n \leq 3$): la recursión resta 4.

Ahora sí: $T(n) = 2 T(n - 4)$, con rigor

El plan para hacerla bien:

- 1 **Definir T con su dominio:** sea $T : \mathbb{N} \rightarrow \mathbb{N}$ tal que $T(n) = 2 T(n - 4)$ para todo $n \geq 4$. De $T(0), \dots, T(3)$ no sabemos nada: sea $a = \max(T(0), T(1), T(2), T(3))$.
- 2 **Definir la propiedad:** $P(n) : T(n) \leq a \cdot 2^{\frac{n}{4}}$.
- 3 **Cuatro casos base** ($0 \leq n \leq 3$): la recursión resta 4.
- 4 **Paso inductivo** (inducción fuerte): para $n \geq 4$, $0 \leq n - 4 < n$ legitima usar $P(n - 4)$.

Ahora sí: $T(n) = 2 T(n - 4)$, con rigor

El plan para hacerla bien:

- 1 **Definir T con su dominio:** sea $T : \mathbb{N} \rightarrow \mathbb{N}$ tal que $T(n) = 2 T(n - 4)$ para todo $n \geq 4$. De $T(0), \dots, T(3)$ no sabemos nada: sea $a = \max(T(0), T(1), T(2), T(3))$.
- 2 **Definir la propiedad:** $P(n) : T(n) \leq a \cdot 2^{\frac{n}{4}}$.
- 3 **Cuatro casos base** ($0 \leq n \leq 3$): la recursión resta 4.
- 4 **Paso inductivo** (inducción fuerte): para $n \geq 4$, $0 \leq n - 4 < n$ legitima usar $P(n - 4)$.
- 5 **Concluir con la definición de O :** exhibir constantes concretas c y n_0 .

Ahora sí: $T(n) = 2 T(n - 4)$, con rigor

El plan para hacerla bien:

- 1 **Definir T con su dominio:** sea $T : \mathbb{N} \rightarrow \mathbb{N}$ tal que $T(n) = 2 T(n - 4)$ para todo $n \geq 4$. De $T(0), \dots, T(3)$ no sabemos nada: sea $a = \max(T(0), T(1), T(2), T(3))$.
- 2 **Definir la propiedad:** $P(n) : T(n) \leq a \cdot 2^{\frac{n}{4}}$.
- 3 **Cuatro casos base** ($0 \leq n \leq 3$): la recursión resta 4.
- 4 **Paso inductivo** (inducción fuerte): para $n \geq 4$, $0 \leq n - 4 < n$ legitima usar $P(n - 4)$.
- 5 **Concluir con la definición de O :** exhibir constantes concretas c y n_0 .

→ Demostración completa en el pizarrón.

Ahora sí: $T(n) = 2 T(n - 4)$, con rigor

El plan para hacerla bien:

- 1 **Definir T con su dominio:** sea $T : \mathbb{N} \rightarrow \mathbb{N}$ tal que $T(n) = 2 T(n - 4)$ para todo $n \geq 4$. De $T(0), \dots, T(3)$ no sabemos nada: sea $a = \max(T(0), T(1), T(2), T(3))$.
- 2 **Definir la propiedad:** $P(n) : T(n) \leq a \cdot 2^{\frac{n}{4}}$.
- 3 **Cuatro casos base** ($0 \leq n \leq 3$): la recursión resta 4.
- 4 **Paso inductivo** (inducción fuerte): para $n \geq 4$, $0 \leq n - 4 < n$ legitima usar $P(n - 4)$.
- 5 **Concluir con la definición de O :** exhibir constantes concretas c y n_0 .

→ Demostración completa en el pizarrón.

Mismo resultado que el alumno del principio, pero ahora **sin baches**: dominio claro, cuatro casos base, HI explícita, y la definición de O aplicada con constantes concretas.

¿Y si mi objeto tiene dos «tamaños»? Inducción en tuplas

A veces el estado natural del problema es un **par** (a, b) : una celda de una matriz, dos índices de una recursión, ... ¿En qué hacemos inducción?

¿Y si mi objeto tiene dos «tamaños»? Inducción en tuplas

A veces el estado natural del problema es un **par** (a, b) : una celda de una matriz, dos índices de una recursión, ... ¿En qué hacemos inducción?

Necesitamos un orden bien fundado

Un orden $\prec$ sobre los pares tal que **no haya cadenas infinitas decrecientes**. Entonces podemos usar inducción fuerte: para probar $P(a, b)$ podemos asumir $P(a', b')$ para todo $(a', b') \prec (a, b)$.

¿Y si mi objeto tiene dos «tamaños»? Inducción en tuplas

A veces el estado natural del problema es un **par** (a, b) : una celda de una matriz, dos índices de una recursión, ... ¿En qué hacemos inducción?

Necesitamos un orden bien fundado

Un orden $\prec$ sobre los pares tal que **no haya cadenas infinitas decrecientes**. Entonces podemos usar inducción fuerte: para probar $P(a, b)$ podemos asumir $P(a', b')$ para todo $(a', b') \prec (a, b)$.

Dos elecciones habituales:

¿Y si mi objeto tiene dos «tamaños»? Inducción en tuplas

A veces el estado natural del problema es un **par** (a, b) : una celda de una matriz, dos índices de una recursión, ... ¿En qué hacemos inducción?

Necesitamos un orden bien fundado

Un orden $\prec$ sobre los pares tal que **no haya cadenas infinitas decrecientes**. Entonces podemos usar inducción fuerte: para probar $P(a, b)$ podemos asumir $P(a', b')$ para todo $(a', b') \prec (a, b)$.

Dos elecciones habituales:

- **Por una medida:** $(a', b') \prec (a, b)$ si $a' + b' < a + b$ (o si $b' < b$, si la recursión sólo achica la segunda componente). Reduce todo a inducción en un natural.

¿Y si mi objeto tiene dos «tamaños»? Inducción en tuplas

A veces el estado natural del problema es un **par** (a, b) : una celda de una matriz, dos índices de una recursión, ... ¿En qué hacemos inducción?

Necesitamos un orden bien fundado

Un orden $\prec$ sobre los pares tal que **no haya cadenas infinitas decrecientes**. Entonces podemos usar inducción fuerte: para probar $P(a, b)$ podemos asumir $P(a', b')$ para todo $(a', b') \prec (a, b)$.

Dos elecciones habituales:

- **Por una medida:** $(a', b') \prec (a, b)$ si $a' + b' < a + b$ (o si $b' < b$, si la recursión sólo achica la segunda componente). Reduce todo a inducción en un natural.
- **Lexicográfico:** $(a', b') \prec (a, b)$ si $a' < a$, o si $a' = a$ y $b' < b$.

¿Y si mi objeto tiene dos «tamaños»? Inducción en tuplas

A veces el estado natural del problema es un **par** (a, b) : una celda de una matriz, dos índices de una recursión, ... ¿En qué hacemos inducción?

Necesitamos un orden bien fundado

Un orden $\prec$ sobre los pares tal que **no haya cadenas infinitas decrecientes**. Entonces podemos usar inducción fuerte: para probar $P(a, b)$ podemos asumir $P(a', b')$ para todo $(a', b') \prec (a, b)$.

Dos elecciones habituales:

- **Por una medida:** $(a', b') \prec (a, b)$ si $a' + b' < a + b$ (o si $b' < b$, si la recursión sólo achica la segunda componente). Reduce todo a inducción en un natural.
- **Lexicográfico:** $(a', b') \prec (a, b)$ si $a' < a$, o si $a' = a$ y $b' < b$.

Lo importante: **definir el orden explícitamente** antes de largar la inducción.

Correctitud de algoritmos recursivos

Si nuestro algoritmo es recursivo, en general vamos a usar **inducción** para probar su correctitud:

Correctitud de algoritmos recursivos

Si nuestro algoritmo es recursivo, en general vamos a usar **inducción** para probar su correctitud:

- 1 Definir una noción de **tamaño** de la entrada (un natural que decrece en cada llamada recursiva).

Correctitud de algoritmos recursivos

Si nuestro algoritmo es recursivo, en general vamos a usar **inducción** para probar su correctitud:

- 1 Definir una noción de **tamaño** de la entrada (un natural que decrece en cada llamada recursiva).
- 2 Definir $P(n)$: «para toda entrada de tamaño n , el algoritmo devuelve lo correcto».

Correctitud de algoritmos recursivos

Si nuestro algoritmo es recursivo, en general vamos a usar **inducción** para probar su correctitud:

- 1 Definir una noción de **tamaño** de la entrada (un natural que decrece en cada llamada recursiva).
- 2 Definir $P(n)$: «para toda entrada de tamaño n , el algoritmo devuelve lo correcto».
- 3 Probar P por inducción (casi siempre fuerte: las llamadas recursivas son a tamaños *menores*, no necesariamente a $n - 1$).

Correctitud de algoritmos recursivos

Si nuestro algoritmo es recursivo, en general vamos a usar **inducción** para probar su correctitud:

- 1 Definir una noción de **tamaño** de la entrada (un natural que decrece en cada llamada recursiva).
- 2 Definir $P(n)$: «para toda entrada de tamaño n , el algoritmo devuelve lo correcto».
- 3 Probar P por inducción (casi siempre fuerte: las llamadas recursivas son a tamaños *menores*, no necesariamente a $n - 1$).

Veamos el ejemplo de exponenciación rápida.

Exponenciación rápida, recursiva

Sean $a \in \mathbb{N}$ y $n \in \mathbb{N}$.

Exponenciación rápida, recursiva

Sean $a \in \mathbb{N}$ y $n \in \mathbb{N}$.

```
procedure Exp( $a$ ,  $n$ )  
  if  $n = 0$  then  
     $\sqsubset$  return 1  
   $b \leftarrow \text{Exp}(a, \lfloor \frac{n}{2} \rfloor)$  ;  
   $c \leftarrow b^2$  ;  
  if  $n \bmod 2 = 1$  then  
     $\sqsubset$   $c \leftarrow c \times a$   
   $\sqsubset$  return  $c$ 
```

Exponenciación rápida, recursiva

Sean $a \in \mathbb{N}$ y $n \in \mathbb{N}$.

```
procedure Exp( $a$ ,  $n$ )  
  if  $n = 0$  then  
     $\sqsubset$  return 1  
   $b \leftarrow \text{Exp}(a, \lfloor \frac{n}{2} \rfloor)$  ;  
   $c \leftarrow b^2$  ;  
  if  $n \bmod 2 = 1$  then  
     $\sqsubset$   $c \leftarrow c \times a$   
   $\sqsubset$  return  $c$ 
```

¿En qué hacemos inducción? La llamada recursiva es con $\lfloor \frac{n}{2} \rfloor < n$ (para $n \geq 1$): inducción fuerte en n , el exponente.

Exponenciación rápida: el plan

- 1 **La propiedad habla del exponente:** $P(n)$: *para todo $a \in \mathbb{N}$, $\text{Exp}(a, n) = a^n$* . El $\forall a$ va *adentro* de $P(n)$: la HI sirve para cualquier base.

Exponenciación rápida: el plan

- 1 **La propiedad habla del exponente:** $P(n)$: *para todo* $a \in \mathbb{N}$, $\text{Exp}(a, n) = a^n$. El $\forall a$ va *adentro* de $P(n)$: la HI sirve para cualquier base.
- 2 **Caso base** $P(0)$: el primer *if*.

Exponenciación rápida: el plan

- 1 **La propiedad habla del exponente:** $P(n)$: para todo $a \in \mathbb{N}$, $\text{Exp}(a, n) = a^n$. El $\forall a$ va adentro de $P(n)$: la HI sirve para cualquier base.
- 2 **Caso base** $P(0)$: el primer *if*.
- 3 **Paso inductivo** con inducción fuerte: usamos la HI en $\lfloor \frac{n}{2} \rfloor < n$, no en $n - 1$.

Exponenciación rápida: el plan

- 1 **La propiedad habla del exponente:** $P(n)$: para todo $a \in \mathbb{N}$, $\text{Exp}(a, n) = a^n$. El $\forall a$ va adentro de $P(n)$: la HI sirve para cualquier base.
- 2 **Caso base** $P(0)$: el primer *if*.
- 3 **Paso inductivo** con inducción fuerte: usamos la HI en $\lfloor \frac{n}{2} \rfloor < n$, no en $n - 1$.
- 4 **Partir en casos** según la paridad de n , igual que el algoritmo (el segundo *if*).

Exponenciación rápida: el plan

- 1 **La propiedad habla del exponente:** $P(n)$: para todo $a \in \mathbb{N}$, $\text{Exp}(a, n) = a^n$. El $\forall a$ va adentro de $P(n)$: la HI sirve para cualquier base.
- 2 **Caso base** $P(0)$: el primer *if*.
- 3 **Paso inductivo** con inducción fuerte: usamos la HI en $\lfloor \frac{n}{2} \rfloor < n$, no en $n - 1$.
- 4 **Partir en casos** según la paridad de n , igual que el algoritmo (el segundo *if*).

→ Demostración en el pizarrón.

Exponenciación rápida: el plan

- 1 **La propiedad habla del exponente:** $P(n)$: para todo $a \in \mathbb{N}$, $\text{Exp}(a, n) = a^n$. El $\forall a$ va adentro de $P(n)$: la HI sirve para cualquier base.
- 2 **Caso base** $P(0)$: el primer *if*.
- 3 **Paso inductivo** con inducción fuerte: usamos la HI en $\lfloor \frac{n}{2} \rfloor < n$, no en $n - 1$.
- 4 **Partir en casos** según la paridad de n , igual que el algoritmo (el segundo *if*).

→ Demostración en el pizarrón.

Los algoritmos recursivos suelen ser más fáciles de demostrar correctos que los iterativos (que necesitan invariantes): no hay estado que se modifica, sólo llamadas a subproblemas menores.

De la demostración al código (y a los tests)

Una fórmula cerrada **demostrada** es un oráculo perfecto para testear la versión recursiva (y viceversa):

```
long long suma_geom(int n) {                //  $3^0 + 3^1 + \dots + 3^n$ 
    if (n == 0) return 1;
    return suma_geom(n - 1) + pot3(n); //  $pot3(n) = 3^n$ 
}
void test() {
    for (int n = 0; n <= 30; ++n) {
        // formula cerrada demostrada hoy:  $(3^{(n+1)} - 1) / 2$ 
        assert(suma_geom(n) == (pot3(n + 1) - 1) / 2);
    }
}
```

De la demostración al código (y a los tests)

Una fórmula cerrada **demostrada** es un oráculo perfecto para testear la versión recursiva (y viceversa):

```
long long suma_geom(int n) {                //  $3^0 + 3^1 + \dots + 3^n$ 
    if (n == 0) return 1;
    return suma_geom(n - 1) + pot3(n); //  $pot3(n) = 3^n$ 
}
void test() {
    for (int n = 0; n <= 30; ++n) {
        // formula cerrada demostrada hoy:  $(3^{(n+1)} - 1) / 2$ 
        assert(suma_geom(n) == (pot3(n + 1) - 1) / 2);
    }
}
```

- El test compara **dos caminos independientes** hacia el mismo valor: si difieren, algo está mal (el código... o la demostración).

De la demostración al código (y a los tests)

Una fórmula cerrada **demostrada** es un oráculo perfecto para testear la versión recursiva (y viceversa):

```
long long suma_geom(int n) {                //  $3^0 + 3^1 + \dots + 3^n$ 
    if (n == 0) return 1;
    return suma_geom(n - 1) + pot3(n); //  $pot3(n) = 3^n$ 
}
void test() {
    for (int n = 0; n <= 30; ++n) {
        // formula cerrada demostrada hoy:  $(3^{(n+1)} - 1) / 2$ 
        assert(suma_geom(n) == (pot3(n + 1) - 1) / 2);
    }
}
```

- El test compara **dos caminos independientes** hacia el mismo valor: si difieren, algo está mal (el código... o la demostración).
- Cuidado con los límites del tipo de dato: 3^{31} no entra en un `int`.

Contradicción y contrarrecíproco

Dos herramientas para cuando el camino directo no sale:

Contrarrecíproco

Probar $P \Rightarrow Q$ es equivalente a probar $\neg Q \Rightarrow \neg P$.

Contradicción y contrarrecíproco

Dos herramientas para cuando el camino directo no sale:

Contrarrecíproco

Probar $P \Rightarrow Q$ es equivalente a probar $\neg Q \Rightarrow \neg P$.

Ejemplo

Si n^2 es par, entonces n es par.

Demostración. Por contrarrecíproco: si n es impar, $n = 2k + 1$ para algún $k \in \mathbb{N}$, entonces $n^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$ es impar. □

Contradicción y contrarrecíproco

Contradicción (absurdo)

Para probar P : asumimos $\neg P$, y derivamos algo falso. **Decir explícitamente** «asumimos por contradicción que. . . », y marcar dónde aparece el absurdo.

Contradicción y contrarrecíproco

Contradicción (absurdo)

Para probar P : asumimos $\neg P$, y derivamos algo falso. **Decir explícitamente** «asumimos por contradicción que. . . », y marcar dónde aparece el absurdo.

Cuidado: en una demostración por contradicción, *todo* lo que deducimos vive bajo una suposición falsa. Ser ordenados es todavía más importante.

El principio del palomar

Principio del palomar (versión simple)

Si repartimos m objetos en n cajas y $m > n$, entonces alguna caja tiene al menos 2 objetos.

El principio del palomar

Principio del palomar (versión simple)

Si repartimos m objetos en n cajas y $m > n$, entonces alguna caja tiene al menos 2 objetos.

Demostración. Por contradicción: supongamos que toda caja tiene a lo sumo 1 objeto. Sea c_i la cantidad de objetos de la caja i . Entonces

$$m = \sum_{i=1}^n c_i \leq \sum_{i=1}^n 1 = n,$$

contradiciendo $m > n$. Luego alguna caja tiene al menos 2 objetos. □

El principio del palomar

Principio del palomar (versión simple)

Si repartimos m objetos en n cajas y $m > n$, entonces alguna caja tiene al menos 2 objetos.

Demostración. Por contradicción: supongamos que toda caja tiene a lo sumo 1 objeto. Sea c_i la cantidad de objetos de la caja i . Entonces

$$m = \sum_{i=1}^n c_i \leq \sum_{i=1}^n 1 = n,$$

contradiciendo $m > n$. Luego alguna caja tiene al menos 2 objetos. □

- La misma idea con promedios: alguna caja tiene al menos $\lceil \frac{m}{n} \rceil$ objetos («no pueden estar todas por debajo del promedio»).

El truco del «primer elemento que cumple»

$\mathbb{N}$ está **bien ordenado**: todo subconjunto no vacío de $\mathbb{N}$ tiene mínimo. Esto habilita un truco que muchas veces *reemplaza* una inducción engorrosa: «*sea i el primer índice tal que...*», «*sea x el mínimo elemento que...*».

El truco del «primer elemento que cumple»

$\mathbb{N}$ está **bien ordenado**: todo subconjunto no vacío de $\mathbb{N}$ tiene mínimo. Esto habilita un truco que muchas veces *reemplaza* una inducción engorrosa: «sea i el primer índice tal que...», «sea x el mínimo elemento que...».

Ejemplo

Todo natural $n \geq 2$ tiene un divisor primo.

Demostración. Sea $D = \{d \in \mathbb{N} \mid d \geq 2, d \mid n\}$. $D \neq \emptyset$ porque $n \in D$. Sea $p = \min D$. Si p no fuera primo, tendría un divisor d con $2 \leq d < p$; pero $d \mid p$ y $p \mid n$ implican $d \mid n$, entonces $d \in D$ y $d < p$, contradiciendo la minimalidad de p . Luego p es primo. $\square$

El truco del «primer elemento que cumple»

$\mathbb{N}$ está **bien ordenado**: todo subconjunto no vacío de $\mathbb{N}$ tiene mínimo. Esto habilita un truco que muchas veces *reemplaza* una inducción engorrosa: «sea i el primer índice tal que...», «sea x el mínimo elemento que...».

Ejemplo

Todo natural $n \geq 2$ tiene un divisor primo.

Demostración. Sea $D = \{d \in \mathbb{N} \mid d \geq 2, d \mid n\}$. $D \neq \emptyset$ porque $n \in D$. Sea $p = \min D$. Si p no fuera primo, tendría un divisor d con $2 \leq d < p$; pero $d \mid p$ y $p \mid n$ implican $d \mid n$, entonces $d \in D$ y $d < p$, contradiciendo la minimalidad de p . Luego p es primo. $\square$

- ¡Hay que justificar que el conjunto es **no vacío**! Si no, el «mínimo» no existe.

El truco del «primer elemento que cumple»

$\mathbb{N}$ está **bien ordenado**: todo subconjunto no vacío de $\mathbb{N}$ tiene mínimo. Esto habilita un truco que muchas veces *reemplaza* una inducción engorrosa: «sea i el primer índice tal que...», «sea x el mínimo elemento que...».

Ejemplo

Todo natural $n \geq 2$ tiene un divisor primo.

Demostración. Sea $D = \{d \in \mathbb{N} \mid d \geq 2, d \mid n\}$. $D \neq \emptyset$ porque $n \in D$. Sea $p = \min D$. Si p no fuera primo, tendría un divisor d con $2 \leq d < p$; pero $d \mid p$ y $p \mid n$ implican $d \mid n$, entonces $d \in D$ y $d < p$, contradiciendo la minimalidad de p . Luego p es primo. $\square$

- ¡Hay que justificar que el conjunto es **no vacío**! Si no, el «mínimo» no existe.
- Adelanto: vuelve todo el tiempo en el resto de la materia («la primera iteración en la que...», «el primer momento en que...»).

Tips para escribir demostraciones

- 1 **Definan $P(n)$ explícitamente**, con todos sus cuantificadores, antes de empezar.

Tips para escribir demostraciones

- 1 **Definan $P(n)$ explícitamente**, con todos sus cuantificadores, antes de empezar.
- 2 Cuenten **cuántos casos base** necesita su paso inductivo (¿usa $P(n - 1)$?
¿ $P(n - 2)$? ¿ $P(n - 4)$?).

Tips para escribir demostraciones

- 1 **Definan $P(n)$ explícitamente**, con todos sus cuantificadores, antes de empezar.
- 2 Cuenten **cuántos casos base** necesita su paso inductivo (¿usa $P(n-1)$?
¿ $P(n-2)$? ¿ $P(n-4)$?).
- 3 **Digan dónde usan la HI**, y verifiquen que el valor donde la usan cae dentro de $\mathbb{N}$ (y del rango donde vale).

Tips para escribir demostraciones

- 1 **Definan $P(n)$ explícitamente**, con todos sus cuantificadores, antes de empezar.
- 2 Cuenten **cuántos casos base** necesita su paso inductivo (¿usa $P(n-1)$?
¿ $P(n-2)$? ¿ $P(n-4)$?).
- 3 **Digan dónde usan la HI**, y verifiquen que el valor donde la usan cae dentro de $\mathbb{N}$ (y del rango donde vale).
- 4 Pónganle **nombre** a todo, **cuantifiquen** todo, y no reusen nombres.

Tips para escribir demostraciones

- 1 **Definan $P(n)$ explícitamente**, con todos sus cuantificadores, antes de empezar.
- 2 Cuenten **cuántos casos base** necesita su paso inductivo (¿usa $P(n-1)$? ¿ $P(n-2)$? ¿ $P(n-4)$?).
- 3 **Digan dónde usan la HI**, y verifiquen que el valor donde la usan cae dentro de $\mathbb{N}$ (y del rango donde vale).
- 4 Pónganle **nombre** a todo, **cuantifiquen** todo, y no reusen nombres.
- 5 Si «sea x el mínimo tal que...»: prueben que el conjunto es no vacío.

Tips para escribir demostraciones

- 1 **Definan $P(n)$ explícitamente**, con todos sus cuantificadores, antes de empezar.
- 2 Cuenten **cuántos casos base** necesita su paso inductivo (¿usa $P(n-1)$? ¿ $P(n-2)$? ¿ $P(n-4)$?).
- 3 **Digan dónde usan la HI**, y verifiquen que el valor donde la usan cae dentro de $\mathbb{N}$ (y del rango donde vale).
- 4 Pónganle **nombre** a todo, **cuantifiquen** todo, y no reusen nombres.
- 5 Si «sea x el mínimo tal que...»: prueben que el conjunto es no vacío.
- 6 Relean su demostración como Beto, el escéptico: ¿en qué oración le mentirían?

Sobre cómo aprender esto

- Nadie aprendió a andar en bicicleta viendo a otros andar. Por cada minuto que pasen leyendo demostraciones, pasen cinco escribiendo las suyas.

Sobre cómo aprender esto

- Nadie aprendió a andar en bicicleta viendo a otros andar. Por cada minuto que pasen leyendo demostraciones, pasen cinco escribiendo las suyas.
- Les va a tomar tiempo. Es frecuente que un ejercicio lleve horas. No está mal: el aprendizaje sucede cuando piensan, intentan, fallan y reflexionan, no cuando «termina el ejercicio».

Sobre cómo aprender esto

- Nadie aprendió a andar en bicicleta viendo a otros andar. Por cada minuto que pasen leyendo demostraciones, pasen cinco escribiendo las suyas.
- Les va a tomar tiempo. Es frecuente que un ejercicio lleve horas. No está mal: el aprendizaje sucede cuando piensan, intentan, fallan y reflexionan, no cuando «termina el ejercicio».
- Si llegaron de P a Q pero no están seguros de que lo que hicieron está bien, **no terminaron el ejercicio**. Si no los convence a ustedes, no va a convencer a ningún par.

Sobre cómo aprender esto

- Nadie aprendió a andar en bicicleta viendo a otros andar. Por cada minuto que pasen leyendo demostraciones, pasen cinco escribiendo las suyas.
- Les va a tomar tiempo. Es frecuente que un ejercicio lleve horas. No está mal: el aprendizaje sucede cuando piensan, intentan, fallan y reflexionan, no cuando «termina el ejercicio».
- Si llegaron de P a Q pero no están seguros de que lo que hicieron está bien, **no terminaron el ejercicio**. Si no los convence a ustedes, no va a convencer a ningún par.
- No acepten algo como cierto sólo porque lo dice un docente (o una IA). Si no lo pueden demostrar, no saben si es cierto.

Sobre cómo aprender esto

- Nadie aprendió a andar en bicicleta viendo a otros andar. Por cada minuto que pasen leyendo demostraciones, pasen cinco escribiendo las suyas.
- Les va a tomar tiempo. Es frecuente que un ejercicio lleve horas. No está mal: el aprendizaje sucede cuando piensan, intentan, fallan y reflexionan, no cuando «termina el ejercicio».
- Si llegaron de P a Q pero no están seguros de que lo que hicieron está bien, **no terminaron el ejercicio**. Si no los convence a ustedes, no va a convencer a ningún par.
- No acepten algo como cierto sólo porque lo dice un docente (o una IA). Si no lo pueden demostrar, no saben si es cierto.

No hay trucos: sólo sudor y tiza.

Bibliografía

- F. Lebrón, *Demostraciones matemáticas*.
- I. S. Sominskii, *El método de la inducción matemática*.
- G. Pólya, *How to Solve It*.
- D. J. Velleman, *How to Prove It*.
- T. Cormen, C. Leiserson, R. Rivest, C. Stein, *Introduction to Algorithms*, caps. 2–4 (análisis de algoritmos, notación asintótica, recurrencias).
- J. Kleinberg, É. Tardos, *Algorithm Design*, cap. 2 (fundamentos de análisis de algoritmos).

¿Preguntas?